

which is just the min of the unary weighted Łukasiewicz t-norm applied to each argument, and which, if all $\beta_i = \beta$, happens to be very similar to ((Sung 1998)). Again using the De Morgan laws, the related binary and n-ary *weighted Gödel t-conorms* are then

$$(\beta_1(x_1^{\oplus w_1}) \vee \beta_2(x_2^{\oplus w_2})) = \min\{1, \max\{0, 1 - \beta_1 + w_1x_1, 1 - \beta_2 + w_2x_2\}\}, \quad (19)$$

$$(\bigvee_{i \in I} \beta_i(x_i^{\oplus w_i})) = \min\{1, \max\{0, 1 - \beta_i + w_ix_i : i \in I\}\} \quad (20)$$

The *weighted Gödel residuum* now solves

$$\max\{z : y \geq \beta_{xy}/w_y (x^{\otimes w_x}/w_y) \wedge \beta_y/w_y (z^{\otimes 1/w_y})\} \quad (21)$$

for $\beta_{xy} = \max\{0, \beta_x + \beta_y - 1\}$ and is given

$$(\beta_x(x^{\otimes w_x}) \Rightarrow \beta_y(y^{\oplus w_y})) = \beta_y(y^{\oplus w_y}) \text{ if } \beta_x(x^{\otimes w_x}) > \beta_y(y^{\oplus w_y}), \text{ else } 1 \quad (22)$$

where operands are again unary weighted Łukasiewicz norms, or specifically

$$\beta_x(x^{\otimes w_x}) = \max\{0, \min\{1, \beta_x - w_x(1 - x)\}\}, \quad (23)$$

$$\beta_y(y^{\oplus w_y}) = \max\{0, \min\{1, 1 - \beta_y + w_y y\}\}. \quad (24)$$

The weighted Gödel norms are commutative if one permutes both weights w_i and biases β_i along with inputs x_i and, similar to the weighted Łukasiewicz norms, are associative if $\beta_i \leq \min\{1, w_i\}$ for each $i \in I$. Likewise, they behave classically for classical input if $1 \leq \beta_i \leq w_i$ for each $i \in I$.

B.3 Parameter semantics

Weights w_i need not sum to 1; accordingly, they are best interpreted as absolute importance as opposed to relative importance. As mentioned above, for conjunctions, increased weight amplifies the respective input’s distance from 1, while for disjunctions, increased weight amplifies the respective input’s distance from 0. Decreased weight has the opposite effect, to the point that inputs with zero weight have no effect on the result at all.

Bias term β is best interpreted as continuously varying the “difficulty” of satisfying the operation. In weighted Łukasiewicz logic, this can so much as translate from one logical connective to another, e.g. from logical AND to logical OR. Constraints imposed on β and w_i can guarantee that the operation performed at each neuron matches the corresponding connective in the represented formula, e.g., when inputs are assumed to be within a given distance of 1 or 0, as further discussed in Section 6.

C The Upward-Downward algorithm (continued)

This section supplements its counterpart in the main paper on page 5, and provides more complete algorithm descriptions. Inference tasks in LNN involve generating proofs of truth values for specified output nodes, given truth values at specified input nodes. Note that a node could both be an input and an output node so that initial proofs can be provided and updated through inference to obtain an aggregate proof as output for the same node. Proof generation operates over a system of formulae that is represented as a mapping of the corresponding syntax tree to a directed graph. An upward pass described in Algorithm 1 calculates a proof at a formula using values from its input terms and subformulae, which corresponds to normal forward computation for neurons. A downward pass detailed in Algorithm 2 proves truth values for each input operand based on the other operand truth values and the enclosing formula’s truth value.

Inference propagates information through edges of the formula syntax tree until convergence where no new proofs are generated, as shown in Algorithm 3. Information flow in connected components can be optimized through sequential traversal over adjacent nodes such that inference has access to the newest proofs recently calculated for neighboring formulae. Preorder and postorder traversals avoid cycles within an inference epoch so that only a single upward or downward calculation is performed for a node within an epoch. Sequential traversal with upward passes starts from propositions, known truth values, or syntax tree leaves, while downward passes move information from outer formulae to inner terms.

LNN converts to a directed acyclic graph through cycle-avoidance, and over alternating upward and downward passes the graph is unrolled according to the traversal sequence to resemble a finite impulse

Algorithm 4: Upward pass to infer formula truth value bounds from subformulae bounds.

```

function upwardPass(formula  $z$ ):                                # recursive upward pass inference function
    if  $z = P(v_0, \dots, v_n)$  then                                #  $z$  is an atom or  $n$ -ary predicate
        if uninitialized( $z$ ) then                                # uninitialized truth value
             $(L_z, U_z) := (0, 1)$                                 # unknown under open-world assumption
        return  $(L_z, U_z)$                                         # existing or default truth value bounds

    for subformula  $x_i \in I$  do                                    # every input operand  $x_i$  of  $z$ 
         $(L'_{x_i}, U'_{x_i}) := \text{upwardPass}(x_i)$                 # obtain operand bounds, recurses to leaves
         $(L_{x_i}, U_{x_i}) := \text{aggregate}(x_i, (L'_{x_i}, U'_{x_i}))$     # aggregate new proof

    if  $z = \neg x$  then                                           #  $z$  performs negation
        return  $(1 - U_x, 1 - L_x)$                                 # single input operand  $x$ 
    else if  $z = x^{\otimes w_x} \rightarrow y^{\oplus w_y}$  then                #  $z$  performs implication
        return  $(\beta(U_x^{\otimes w_x} \rightarrow L_y^{\oplus w_y}), \beta(L_x^{\otimes w_x} \rightarrow U_y^{\oplus w_y}))$     # two input operands  $x, y$ 
    else if  $z = \beta(\bigotimes_{i \in I} x_i^{\otimes w_i})$  then                #  $z$  performs conjunction
        return  $(\beta(\bigotimes_{i \in I} L_{x_i}^{\otimes w_i}), \beta(\bigotimes_{i \in I} U_{x_i}^{\otimes w_i}))$     # multi-input conjunction
    else if  $z = \beta(\bigoplus_{i \in I} x_i^{\oplus w_i})$  then                #  $z$  performs disjunction
        return  $(\beta(\bigoplus_{i \in I} L_{x_i}^{\oplus w_i}), \beta(\bigoplus_{i \in I} U_{x_i}^{\oplus w_i}))$     # multi-input disjunction
    else if  $z = \forall_{g \in G} x(g)$  then                            #  $z$  is universal quantifier over groundings  $G$ 
        return  $(\min_{g \in G} L_{x(g)}, \min_{g \in G} U_{x(g)})$         # universal quantification over  $G$ 
    else if  $z = \exists_{g \in G} x(g)$  then                            #  $z$  is existential quantifier over groundings  $G$ 
        return  $(\max_{g \in G} L_{x(g)}, \max_{g \in G} U_{x(g)})$         # existential quantification over  $G$ 

function aggregate(formula  $z, (L'_z, U'_z)$ ):                    # aggregate offered proof  $(L'_z, U'_z)$  for  $z$ 
    return  $(\max(L_z, L'_z), \min(U_z, U'_z))$                     # monotonically tighten existing  $(L_z, U_z)$  bounds

```

recurrent network. However, given the monotonic tightening of proof aggregation, the network is more constrained than the dynamic behavior exhibited by recurrent neural networks. Downward traversal can mirror the upward pass, although transient directional edges are effectively introduced by downward inference so separate proofs are offered to each neuron input according to the functional inverse calculation with analytically transformed weights.

Given both upward and downward computations for each connective, overall computation is characterized as follows:

1. Initialize neurons corresponding to propositions and formula roots with starting truth value bounds. Usually, all formulae are initialized as true. Propositions with starting bounds represent input data.
2. For each formula, evaluate neurons in the forward direction in a pass from leaves to root, storing computed bounds at each node. Then, backtrack to each leaf using inverse computations to update subformula bounds based on stored bounds (and formula initial bounds).
3. Aggregate the tightest bounds computed at leaves for each proposition.
4. Return to step 2 until bounds converge. Oscillation cannot occur because bounds tightening is monotonic.
5. Inspect computed bounds at specific propositions or formulae, i.e. those representing the predictions of the model.

As suggested in step 3, one may simply use min and max to aggregate upper and lower bounds proved for each proposition, though smoothed versions of these may be preferred to spread gradient information over multiple proofs. Alternately, when targeting classical logic, one can use conjunction and disjunction (themselves possibly smoothed) to aggregate proposition bounds. When doing so, there is an opportunity to reuse propositions' weights from their respective proofs, so as to limit the effect of proofs in which the proposition only plays a minor role.

As suggested in step 5, prediction results are obtained by inspecting the outputs of one or more neurons, similar to what would be done for a conventional neural network. Different, however, is the fact that different neurons may serve as inputs and results for different queries, indeed with a

Algorithm 5: Downward pass to infer subformula truth value bounds.

```

function downwardPass(formula  $z$ ):           # recursive downward pass inference function
    for subformula  $x_j \in I$  do                 # every input operand  $x_j$  of  $z$ 
        if  $z = \neg x$  then                     #  $z$  performs negation
             $(L'_x, U'_x) := (1 - U_z, 1 - L_z)$  # proof for single input operand  $x$ 
        else if  $z = \beta(x^{\otimes w_x} \rightarrow y^{\oplus w_y})$  then #  $z$  performs implication
             $L'_x := \begin{cases} \beta/w_x (U_z^{\otimes 1/w_x} \rightarrow L_y^{\oplus w_y/w_x}) & \text{if } U_z < 1, \\ 0 & \text{otherwise.} \end{cases}$  # left input  $x$  lower bound
             $U'_x := \begin{cases} \beta/w_x (L_z^{\otimes 1/w_x} \rightarrow U_y^{\oplus w_y/w_x}) & \text{if } L_z > 0, \\ 1 & \text{otherwise} \end{cases}$  # left input  $x$  upper bound
             $L'_y := \begin{cases} \beta/w_y (L_x^{\otimes w_x/w_y} \otimes L_z^{\otimes 1/w_y}) & \text{if } L_z > 0, \\ 0 & \text{otherwise} \end{cases}$  # right input  $y$  lower bound
             $U'_y := \begin{cases} \beta/w_y (U_x^{\otimes w_x/w_y} \otimes U_z^{\otimes 1/w_y}) & \text{if } U_z < 1, \\ 1 & \text{otherwise} \end{cases}$  # right input  $y$  upper bound
        else if  $z = \beta(\bigotimes_{i \in I} x_i^{\otimes w_i})$  then #  $z$  performs conjunction
             $L'_{x_j} := \begin{cases} \beta/w_j ((\bigotimes_{i \neq j} U_{x_i}^{\otimes w_i/w_j}) \rightarrow L_z^{\oplus 1/w_j}) & \text{if } L_z > 0, \\ 0 & \text{otherwise,} \end{cases}$  # lower bound proof
             $U'_{x_j} := \begin{cases} \beta/w_j ((\bigotimes_{i \neq j} L_{x_i}^{\otimes w_i/w_j}) \rightarrow U_z^{\oplus 1/w_j}) & \text{if } U_z < 1, \\ 1 & \text{otherwise,} \end{cases}$  # upper bound proof
        else if  $z = \beta(\bigoplus_{i \in I} x_i^{\oplus w_i})$  then #  $z$  performs disjunction
             $L'_{x_j} := \begin{cases} \beta/w_j ((\bigotimes_{i \neq j} (\neg U_{x_i})^{\otimes w_i/w_j}) \otimes L_z^{\otimes 1/w_j}) & \text{if } L_z > 0, \\ 0 & \text{otherwise,} \end{cases}$  # lower bound
            proof
             $U'_{x_j} := \begin{cases} \beta/w_j ((\bigotimes_{i \neq j} (\neg L_{x_i})^{\otimes w_i/w_j}) \otimes U_z^{\otimes 1/w_j}) & \text{if } U_z < 1, \\ 1 & \text{otherwise.} \end{cases}$  # upper bound
            proof
        else if  $z = \forall_{g \in G} x(g)$  then #  $z$  is universal quantifier over groundings  $G$ 
            for  $g \in G$  do # every  $n$ -ary grounding tuple  $g$ 
                 $(L'_{x(g)}, U'_{x(g)}) := (L_z, 1)$  # proof of minimum lower bound
        else if  $z = \exists_{g \in G} x(g)$  then #  $z$  is existential quantifier over groundings  $G$ 
            for  $g \in G$  do # every  $n$ -ary grounding tuple  $g$ 
                 $(L'_{x(g)}, U'_{x(g)}) := (0, U_z)$  # proof of maximum upper bound
         $(L_{x_i}, U_{x_i}) := \text{aggregate}(x_i, (L'_{x_i}, U'_{x_i}))$  # aggregate new proof
        downwardPass( $x_i$ ) # recurses to leaves, stop before cyclic loop

```

Algorithm 6: Recurrent inference procedure with recursive directional graph traversal.

```

function inference(formulae  $\mathbf{z}$ ):           # iterative function with input formulae
    while  $\sum(|\delta L_z| + |\delta U_z|) > \epsilon$  do # new proofs being generated
        for  $r \in \text{roots}(\mathbf{z})$  do # root nodes of system of formulae
             $(L'_r, U'_r) := \text{upwardPass}(r)$  # leaves-to-root traversal
             $(L_r, U_r) := \text{aggregate}(r, (L'_r, U'_r))$  # root aggregates new proof
            downwardPass( $r$ ) # root-to-leaf traversal

```
